```
  </p>
{% endif %}

{% endblock content %}
```

This template contains the by now familiar `{{ form.as_p }}` and black magic security token. The view passes a value in the context dictionary called `validlink`, which is `True` if the one-time token is valid. This allows you to use the same template for the view, even if the user has messed with the token. It does mean you need to have logic for handling the error case, though—not showing the `<form>` tag is good enough.

The final step in the process is telling the user that the reset process is complete. The password reset view expects this content to be in password_reset_complete.html. This can be a simple message, but it is often handy to include a link to the login page as well. Create RiffMates/templates/registration/password_reset_complete.html, using the contents of the following listing.

Listing 6.13 Template telling the user their password reset process is complete

```
{% extends "base.html" %}

{% block content %}
<h2>Your password has been set</h2>

<p><a href="{{ login_url }}">Login</a></p>
{% endblock content %}
```

The URL to the login page does not include the `?next` argument, so if the user clicks it, they are sent to the default. If you added `LOGIN_REDIRECT_URL = '/bands/bands/'` to the configuration, as recommended earlier, that default is the bands listing page.

If you've done all that work, you're ready to test it out. Start the development server, and make sure to log out. Visit one of your new restricted view pages, and you will be redirected to the login page. On the login page, click the Lost Password? link. Unless you had the foresight to include an email address for the *nicky* account, the admin will be the only account with an email value. Use *admin@example.com* in the reset form, and then copy the reset link printed to your console. Paste the link in your browser, and complete the password reset process. And there you go! RiffMates is now a multiuser site, complete with restricted pages and password management.

6.7 Exercises

1. The `@login_required` decorator only checks if a user got authenticated. A more generic view restriction can be performed using the `@user_passes_test` decorator in the same module (https://mng.bz/2KKN). It takes one required argument: a reference to a function that takes the `User` object as an argument and returns `True` if the user should be allowed in. Write a new view `venues_restricted()` that is only visible by those users associated with a `Venue` object and displays a list of all venues. Hint: You can take advantage of other views in your code. They are just functions.